FinServ Lite - UML Class Diagram

Legend

solid line, hollow triangle = extends

solid line, open arrow = association

dashed line, open arrow = uses / returns / throws

\+ public - private # protected ~ package

underline = static *italic* = abstract

0..* = many, 1 = one

Not drawn: menus use util.ConsoleUtil, DAOs use util.ResultSetUtil, every service and DAO throws exception.BankException, Main creates all DAOs, services and menus.

util

«utility»
ConsoleUtil

-SCANNER: Scanner

-ConsoleUtil()
+readLine(String): String
+readInt(String): int
+readDecimal(String): BigDecimal
+readDate(String): LocalDate
+title(String): void
+info(String): void
+error(String): void
+printRows(List<Map<String, Object>>): void

«utility»
ResultSetUtil

-ResultSetUtil()
+toRows(ResultSet): List<Map<String, Object>>

Main

-Main()
+main(String[]): void

creates and starts

ui

MainMenu

-customerMenu: CustomerMenu
-accountMenu: AccountMenu
-transactionMenu: TransactionMenu
-loanMenu: LoanMenu
-reportMenu: ReportMenu
+MainMenu(CustomerMenu, AccountMenu, TransactionMenu, LoanMenu, ReportMenu)
+run(): void

opens

LoanMenu

-loanService: LoanService
+LoanMenu(LoanService)
+run(): void

ReportMenu

-reportService: ReportService
+ReportMenu(ReportService)
+run(): void

AccountMenu

-accountService: AccountService
+AccountMenu(AccountService)
+run(): void
-print(Account): void

TransactionMenu

-transactionService: TransactionService
+TransactionMenu(TransactionService)
+run(): void
-print(Transaction): void

CustomerMenu

-customerService: CustomerService
+CustomerMenu(CustomerService)
+run(): void
-register(): void
-print(Customer): void

calls

service

LoanService

-loanDao: LoanDao
+LoanService(LoanDao)
+apply(int, String, BigDecimal, BigDecimal, int): int
+approve(int): void
+payEmi(int): void
+loansOf(int): List<Map<String, Object>>
+schedule(int): List<Map<String, Object>>

ReportService

-reportDao: ReportDao
-branchDao: BranchDao
+ReportService(ReportDao, BranchDao)
+branches(): List<Branch>
+branchSummary(): List<Map<String, Object>>
+topCustomers(int): List<Map<String, Object>>
+monthlySummary(): List<Map<String, Object>>
+auditTrail(int): List<Map<String, Object>>

AccountService

-accountDao: AccountDao
+AccountService(AccountDao)
+open(int, int, int, BigDecimal): String
+accountsOf(int): List<Account>
+get(String): Account
+close(String): void
+statement(String, LocalDate, LocalDate): List<Map<String, Object>>
+totalBalance(int): BigDecimal

TransactionService

-transactionDao: TransactionDao
+TransactionService(TransactionDao)
+deposit(String, BigDecimal, String): void
+withdraw(String, BigDecimal, String): void
+transfer(String, String, BigDecimal, String): void
+recent(String, int): List<Transaction>
-requirePositive(BigDecimal): void

CustomerService

XYC_VALUES: Set<String>
-customerDao: CustomerDao
+CustomerService(CustomerDao)
+register(String, String, String, String, LocalDate, String): int
+listAll(): List<Customer>
+get(int): Customer
+changeKyc(int, String): void
+search(String, String): List<Map<String, Object>>

uses

dao (data access: all SQL lives here)

LoanDao

-SCHEDULE_SQL: String
+apply(int, String, BigDecimal, BigDecimal, int): int
+approve(int): void
+payEmi(int): void
+findByCustomer(int): List<Map<String, Object>>
+schedule(int): List<Map<String, Object>>
-callWithLoanId(String, int): void

ReportDao

+topCustomers(int): List<Map<String, Object>>
+monthlySummary(): List<Map<String, Object>>
+auditTrail(int): List<Map<String, Object>>
-query(String, int): List<Map<String, Object>>

BranchDao

+findAll(): List<Branch>
+summary(): List<Map<String, Object>>

AccountDao

-SELECT_ACCOUNT: String
+openAccount(int, int, int, BigDecimal): String
+findByCustomer(int): List<Account>
+findByAccountNo(String): Optional<Account>
+close(String): void
+statement(String, LocalDate, LocalDate): List<Map<String, Object>>
+totalBalance(int): BigDecimal
-setDate(CallableStatement, int, LocalDate): void
-map(ResultSet): Account

TransactionDao

+deposit(String, BigDecimal, String): void
+withdraw(String, BigDecimal, String): void
+transfer(String, String, BigDecimal, String): void
+recent(String, int): List<Transaction>
-call(String, String, BigDecimal, String): void

CustomerDao

-COLUMNS: String
+findAll(): List<Customer>
+findById(int): Optional<Customer>
+insert(String, String, String, String, LocalDate, String): int
+updateKycStatus(int, String): boolean
+search(String, String): List<Map<String, Object>>
-map(ResultSet): Customer

«abstract»
BaseDao

#connection(): Connection
#wrap(SQLException): BankException

returns

config

«singleton»
DatabaseConfig

-instance: DatabaseConfig
-url: String
-user: String
-password: String
-DatabaseConfig()
+getInstance(): DatabaseConfig
+getConnection(): Connection
-env(String, String): String

opens connections via

exception

BankException

-serialVersionUID: long
+BankException(String)
+BankException(String, Throwable)

throws

«java.lang»
RuntimeException

model

«record»
Branch

-id: int
-name: String
-ifscCode: String
-city: String
-active: boolean
+Branch(int, String, String, String, boolean)

«record»
Account

-id: int
-accountNo: String
-customerId: int
-typeName: String
-branchName: String
-balance: BigDecimal
-status: String
+Account(int, String, int, String, String, BigDecimal, String)

«record»
Customer

-id: int
-fullName: String
-email: String
-phone: String
-dateOfBirth: LocalDate
-kycStatus: String
+Customer(int, String, String, String, LocalDate, String)

«record»
Loan

-id: int
-customerId: int
-principal: BigDecimal
-annualRate: BigDecimal
-months: int
-emi: BigDecimal
-status: String
+Loan(int, int, BigDecimal, BigDecimal, int, BigDecimal, String)

«record»
Transaction

-id: long
-time: LocalDateTime
-type: String
-amount: BigDecimal
-balanceAfter: BigDecimal
-description: String
+Transaction(long, LocalDateTime, String, BigDecimal, BigDecimal, String)

returns

0..*